# Práctica Número 3

## Semántica

**Objetivo**: Interpretar el concepto de semántica de los lenguajes de programación.

**Ejercicio 1**: ¿Qué define la semántica?

La semántica describe el significado de los símbolos, palabras y frases de un lenguaje ya sea lenguaje natural o informático. Describe el significado de programas sintácticamente válidos para un lenguaje determinado. Así se da significado a las construcciones del lenguaje.

**Ejercicio 2**:

a. ¿Qué significa compilar un programa?

Compilar un programa es el proceso de traducir el código fuente escrito en un lenguaje de alto nivel a un lenguaje de bajo nivel (generalmente código máquina), de manera que pueda ser ejecutado por la computadora.

b. Describa brevemente cada uno de los pasos necesarios para compilar un programa.

- Análisis léxico: se leen los caracteres del código fuente y se agrupan en tokens (palabras clave, identificadores, operadores, etc.).
- Análisis sintáctico: se verifica que los tokens estén organizados según las reglas gramaticales del lenguaje (estructura correcta del programa).
- Análisis semántico: se comprueba que las construcciones tengan sentido (por ejemplo, tipos compatibles, variables declaradas, uso correcto de funciones).
- Generación de código intermedio (opcional según el compilador): se traduce el programa a una representación intermedia más fácil de optimizar.
- Optimización: se mejora el código intermedio para hacerlo más eficiente.
- Generación de código final: se traduce al código máquina o ensamblador que puede ejecutar la computadora.

c. ¿En qué paso interviene la semántica y cuál es su importancia dentro de la compilación?

La semántica interviene en el análisis semántico. Su importancia radica en que verifica que el programa tenga sentido, más allá de estar bien escrito sintácticamente. Permite detectar errores como uso de variables no declaradas, incompatibilidad de tipos o llamadas incorrectas a funciones, evitando que programas incorrectos lleguen a ejecutarse.

**Ejercicio 3**: Con respecto al punto anterior ¿es lo mismo compilar un programa que interpretarlo? Justifique su respuesta mostrando las diferencias básicas, ventajas y desventajas de cada uno.

No, no es lo mismo compilar que interpretar un programa, puesto que **compilar** traduce todo el programa a código máquina antes de ejecutarlo; pero **interpretar** ejecuta el programa línea por línea sin generar un ejecutable previo.

| | **VENTAJAS** | **DESVENTAJAS** |
|---|---|---|
| ***COMPILACIÓN*** | • Mayor velocidad de ejecución (ya está traducido a código máquina).<br>• Los errores se detectan antes de ejecutar el programa. | • Requiere más tiempo inicial (hay que compilar todo).<br>• Menor portabilidad (el ejecutable depende de la máquina/arquitectura). |
| ***INTERPRETACIÓN*** | • Mayor portabilidad (mismo código se ejecuta en otras máquinas con el intérprete).<br>• Permite probar y ejecutar rápido (útil para desarrollo y testing). | • Menor velocidad de ejecución (traduce mientras ejecuta).<br>• Los errores pueden aparecer en tiempo de ejecución. |

En conclusión, no son lo mismo porque la compilación traduce todo el programa antes de ejecutarlo, mientras que la interpretación lo ejecuta directamente. La compilación es más eficiente en ejecución, mientras que la interpretación es más flexible y portable.

**Ejercicio 4**: Explique claramente la diferencia entre un error sintáctico y uno semántico. Ejemplifique cada caso.

Un **error sintáctico** ocurre cuando el programa no respeta las reglas gramaticales del lenguaje (la forma en que debe escribirse el código). Estos errores son detectados por el analizador sintáctico del

lenguaje (parser) durante la compilación o interpretación. Un **error semántico** ocurre cuando el programa está bien escrito sintácticamente, pero no tiene sentido o produce un comportamiento incorrecto. Por ejemplo:

- En java: *int x = 5* **Error sintáctico → Faltan ; al final**.
  *int x = "Mar"* **Error semántico → La sintaxis es correcta, pero no puedo asignar un String a un entero**.

**Ejercicio 5**: Sean los siguientes ejemplos de programas. Analice y diga qué tipo de error se produce (Semántico o Sintáctico) y en qué momento se detectan dichos errores (Compilación o Ejecución). Aclaración: Los valores de la ayuda pueden ser mayores.

- Pascal

```
Program P
var 5: integer; → 5 no es un nombre válido para una variable (sintáctico).
var a:char;
Begin
    for i:=5 to 10 do begin → la variable i no está declarada (semántico).
        write(a); → a no tiene valor asignado (semántico).
        a=a+1; → no puedo sumar un int a un char (semántico). La asignación es := (sintáctico).
    end;
End.
```

*Ayuda: Sintáctico 2, Semántico 3*

- Java

```
public String tabla(int numero, arrayList listado) → es ArrayList (sintáctico)
    {
        String result = null;
        for(i = 1; i < 11; i--) { → la variable i no está declarada (semántico). i—(lógico).
            result += numero + "x" + i + "=" + (i*numero) + "\n"; → no puedo concatenar algo con valor null (semántico).
            listado.get(listado.size()-1)=(BOOLEAN) numero>i; → get devuelve un valor, tendría que ser .set, además es boolean o Boolean (sintácticox2). Mal casteado número,
```

de int a Boolean no se puede (semántico).

}

return true; → Debería devolver un String (semántico).

}

*Ayuda: Sintácticos 4, Semánticos 3, Lógico 1*

- C

```
# include <stdio.h>
int suma; /* Esta es una variable global */
int main()
{ int indice;
        encabezado; → Identificador sin declarar ni definir (semántico).
        for (indice = 1 ; indice <= 7 ; indice ++)
        cuadrado (indice); → Función usada sin declaración previa (semántico).
        final(); Llama a la función final */ → Comentario mal formado, falta /* (sintáctico). Llama a una función no definida (semántico).
        return 0;
}
cuadrado (numero) → Función mal definida, debería ser void cuadrado(int numero) (sintáctico).
int numero;
{ int numero_cuadrado;
        numero_cuadrado == numero * numero; → == no asigna, compara (semántico).
        suma += numero_cuadrado; → la variable suma no se inicializa (semántico).
        printf("El cuadrado de %d es %d\n",
        numero, numero_cuadrado);
}
```

*Ayuda: Sintácticos 2, Semánticos 6*

- Python

```
#!/usr/bin/python
print "\nDEFINICION DE NUMEROS PRIMOS"
r = 1
```

```
while r = True: → debería ser == (sintáctico).
    N = input("\nDame el numero a analizar: ") → input puede devolver string y lo usa como número (semántico).
    i = 3
    fact = 0
    if (N mod 2 == 0) and (N != 2): → mod no es válido en Phyton (sintáctico).
        print "\nEl numero %d NO es primo\n" % N
    else:
        while i <= (N^0.5): → ^ no es potencia en Phyton (semántico).
            if (N % i) == 0:
                mensaje="\nEl numero ingresado NO es primo\n" % N → se usa % N pero el string no tiene ningún placeholder (%d).
                msg = mensaje[4:6]
                print msg
                fact = 1
            i+=2
        if fact == 0:
            print "\nEl numero %d SI es primo\n" % N
    r = input("Consultar otro número? SI (1) o NO (0)--->> ")
```

*Ayuda: Sintácticos 2, Semánticos 3*

- Ruby

```
def ej1
    Puts 'Hola, ¿Cuál es tu nombre?'
    nom = gets.chomp
    puts 'Mi nombre es ', + nom
    puts 'Mi sobrenombre es 'Juan''
    puts 'Tengo 10 años'
    meses = edad*12 → edad no fue definida antes de usarla (semántico).
    dias = 'meses' *30 → multiplicas un string en vez de la variable meses (semántico).
    hs= 'dias * 24' → no es un cálculo, es un string (semántico).
```

```
    puts 'Eso es: meses + ' meses o ' + dias + ' días o ' + hs + ' horas'
    puts 'vos cuántos años tenés'
    edad2 = gets.chomp
    edad = edad + edad2.to_i
    puts 'entre ambos tenemos ' + edad + ' años'
    puts '¿Sabes que hay ' + name.length.to_s + ' caracteres en tu nombre, ' + name + '?' → la variable name no existe (semántico).
end
```

*Ayuda: Semánticos +4*

**Ejercicio 5**: Dado el siguiente código escrito en pascal. Transcriba la misma funcionalidad de acuerdo al lenguaje que haya cursado en años anteriores. Defina brevemente la sintaxis (sin hacer la gramática) y semántica para la utilización de arreglos y estructuras de control del ejemplo.

```
Procedure ordenar_arreglo(var arreglo: arreglo_de_caracteres;cont:integer);
var
    i:integer; ordenado:boolean;
    aux:char;
begin
    repeat
    ordenado:=true;
    for i:=1 to cont-1 do
    if ord(arreglo[i])>ord(arreglo[i+1])
        then begin
            aux:=arreglo[i];
            arreglo[i]:=arreglo[i+1];
            arreglo[i+1]:=aux; ordenado:=false
            end;
    until ordenado;
end;
```

*Observación: Aquí sólo se debe definir la instrucción y qué es lo que hace cada una; detallando alguna*

*particularidad del lenguaje respecto de ella. Por ejemplo, el for de java necesita definir una variable entera, una condición y un incremento para dicha variable.*

- **Arreglos**

Sintaxis: se declaran como char[]arreglo;

Semántica: estructura que almacena valores del mismo tipo en posiciones consecutivas de memoria. Se accede mediante un índice a cada elemento (en java el primer elemento está en i=0).

```java
public static void ordenarArreglo(char[] arreglo, int cont) {
    boolean ordenado;
    char aux;

    do {
        ordenado = true;
        for (int i = 0; i < cont - 1; i++) {
            if (arreglo[i] > arreglo[i + 1]) {
                aux = arreglo[i];
                arreglo[i] = arreglo[i + 1];
                arreglo[i + 1] = aux;
                ordenado = false;
            }
        }
    } while (!ordenado);
}
```

- **Estructura for**

Sintaxis: for(inicialización; condición, incremento)

Semántica: permite repetir un bloque de código una cantidad determinada de veces.

- **Estructura do-while**

Sintaxis: do {…} while (condición);

Semántica: ejecuta el bloque al menos una vez y luego repite mientras la condición sea verdadera.

- **if**

Sintaxis: if (condición) {…}

Semántica: evalúa una condición y ejecuta el bloque entre llaves si es verdadera.

**Ejercicio 6**: Explique cuál es la semántica para las variables predefinidas en lenguaje Ruby self y nil. ¿Qué valor toman; cómo son usadas por el lenguaje?

**Semántica de self y nil en Ruby**

- ***self***: es una variable predefinida que representa el objeto actual en el contexto de ejecución. Su valor depende de dónde se use:
    - Dentro de un método de instancia, self hace referencia a la instancia del objeto.
    - En un método de una clase, self hace referencia a la clase.
    - En un módulo, hace referencia al módulo.
    - Fuera de cualquier clase o módulo, hace referencia al objeto principal.

Se utiliza para: referirse al propio objeto, llamar a sus métodos, evitar ambigüedades (por ejemplo, diferenciar variables locales de atributos).

- ***nil***: es un objeto especial que representa la ausencia de valor. Es equivalente a "nada" o "vacío". Es una instancia de la clase NilClass. Se usa cuando una variable no tiene valor asignado o un método no retorna nada. En expresiones booleanas se considera falso (junto con false).

**Ejercicio 7**: Determine la semántica de null y undefined para valores en javascript.¿Qué diferencia hay entre ellos?

- ***null***: representa la ausencia intencional de valor. Es un valor que el programador asigna explícitamente para indicar "no hay nada". Es un **objeto primitivo**. Una variable nunca va a ser null a menos que en algún lugar del código el programador lo asigne. Ejemplo: Objeto x = null;
- ***undefined***: representa una variable que ha sido declarada pero no inicializada. Es el valor que JavaScript asigna automáticamente cuando no hay un valor definido. Es un **valor primitivo**, pero no es un objeto. Ejemplo: let x; console.log(x); // undefined (pues aún no le asigne un valor a la variable x).

**Ejercicio 8**: Determine la semántica de la sentencia break en C, PHP, javascript y Ruby. Cite las características más importantes de esta sentencia para cada lenguaje.

La sentencia **break** se utiliza en general para interrumpir la ejecución de una estructura de control (como bucles o switch) y salir inmediatamente de ella.

- C: Interrumpe la ejecución de bucles (for, while, do-while) y de la estructura switch. Siempre sale de la estructura de control más interna. En el caso de switch, se utiliza para evitar que se ejecuten los casos siguientes (fall-through). No permite salir de múltiples niveles de anidamiento.
- PHP: Interrumpe bucles y switch, saliendo de la estructura más interna por defecto. Permite especificar un número para indicar cuántos niveles de estructuras anidadas se desean abandonar (por ejemplo, break 2). Esto lo hace especialmente útil en estructuras anidadas.
- JavaScript: Interrumpe bucles y switch, saliendo de la estructura más interna. Permite el uso de etiquetas (labels) para salir de bloques específicos más allá del nivel inmediato. En switch, se utiliza para evitar la ejecución de los casos siguientes.
- Ruby: Interrumpe la ejecución de bucles como while y for, así como también de iteradores como each. Finaliza el bucle actual inmediatamente. No es necesario ni se utiliza en estructuras

case, ya que estas no presentan comportamiento de caída entre casos.

**Ejercicio 9**: Defina el concepto de ligadura y su importancia respecto de la semántica de un programa. ¿Qué diferencias hay entre ligadura estática y dinámica? Cite ejemplos (proponer casos sencillos).

La **ligadura (binding)** es el proceso mediante el cual se asocia un nombre con una entidad (por ejemplo, una variable con un valor, un tipo, una dirección de memoria o una función). Es fundamental para la semántica de un programa porque permite determinar qué significa cada identificador y cómo se comporta durante la ejecución.

Importancia en la semántica: la ligadura define el significado de los elementos del programa, ya que establece a qué valor, tipo o comportamiento está asociado cada nombre. Sin ligadura, no se podría interpretar correctamente qué hace un programa.

- ***Ligadura estática***: se realiza en tiempo de compilación. Las asociaciones quedan definidas antes de ejecutar el programa. Tiene mayor eficiencia y los errores se detectan de manera temprana. (Ejemplo en Java: *int x = 5;* → El tipo de x se conoce en compilación).
- ***Ligadura dinámica***: Se realiza en tiempo de ejecución. Las asociaciones pueden cambiar mientras el programa se ejecuta. Mayor flexibilidad, pero menor control en compilación. (Ejemplo en Java: *let x = 5; x = "hola";* → El tipo de x cambia en tiempo de ejecución).

En conclusión, la ligadura estática ocurre antes de ejecutar el programa y es fija, mientras que la dinámica ocurre durante la ejecución y puede cambiar.